Full Stack Development using MERN

Smart Swasthya – Community Health Record and Follow-up Management System

Student Name	Prince Tiwari
Roll Number	T046
Class	TYBScIT
Subject	Full Stack Development using MERN
Academic Year	2026-2027
College Name	Sheth L.U.J. College of Arts and Sir M.V. College of Science and Commerce
Department Name	Information Technology
Submitted to	Prof. Rohini Bhosale

1. Project Title

Smart Swasthya – Community Health Record and Follow-up Management System

2. Objective

The main objective of Smart Swasthya is to develop a web-based application for managing community health records in an organized manner. The system helps maintain household and patient information, record health measurements, manage patient visits, track follow-up dates, and maintain preventive healthcare activities. It also provides priority assessment and report generation to support better organization of community health information.

3. Project Description

Smart Swasthya is a web-based community health record management system designed to organize household information, patient details, health records, and follow-up activities.

The application provides a dashboard to view important information and separate modules to manage households and patients. Health records can include weight, blood pressure, temperature, glucose readings, and notes. Patient visits and preventive activities can also be recorded for future reference.

The system includes a priority assessment feature that evaluates selected indicators, such as overdue follow-ups, recorded health readings, patient age, and previous visits. A reports module displays record counts and supports exporting patient information into a CSV file.

The application uses Python for backend processing, SQLite for database storage, HTML for page structure, CSS for styling, and JavaScript for selected frontend interactions.

4. Functional Requirements

Feature	Requirements
Dashboard	Displays an overview of households, patients, visits, and health records.
Household Management	Allows household information, locality, and contact details to be recorded and viewed.
Patient Management	Maintains patient details, including name, age, gender, contact, and household association.

Health Records	Stores health measurements such as weight, blood pressure, temperature, glucose, and notes.
Visit Management	Records visit dates, purpose, notes, and next follow-up dates.
Preventive Activities	Maintains preventive healthcare activity details, dates, status, and notes.
Priority Assessment	Calculates a priority level using selected record indicators and follow-up information.
Reports	Displays counts of important records for project reporting.
CSV Export	Exports selected patient and household details into a CSV file.
Search and Filtering	Provides table filtering where implemented in the interface.

5. Tools and Technologies Used

- Python
- SQLite
- SQL
- HTML
- CSS
- JavaScript
- Visual Studio Code or another suitable code editor
- Git and GitHub

6. Technical Requirements

Component	Requirement
Frontend	HTML and CSS are used to create the web interface and style the pages.
JavaScript	Supports selected interactive functions, including table filtering and chart rendering.
Backend	Python handles HTTP requests, form submissions, application logic, and responses.
Database	SQLite stores household, patient, health record, visit, and preventive activity data.

Database Operations	SQL statements are used to create tables, insert records, retrieve data, and maintain relationships.
Request Handling	Python's HTTP server handles GET and POST requests for application pages and forms.
Priority Assessment	A rule-based scoring mechanism assigns HIGH, MEDIUM, or LOW priority levels.
Report Generation	The reports module displays record counts and provides CSV export functionality.
Data Relationships	Patient records are associated with households, health records, visits, and preventive activities.

7. Validation Requirements

- **Required Fields:** Important fields such as household code, locality, patient name, age, and health record date must be supplied where required.
- **Data Type Validation:** Age, weight, temperature, glucose, and other numeric values should be entered in appropriate formats.
- **Date Validation:** Visit dates and follow-up dates should use valid date formats.
- **Unique Household Code:** Household codes are stored with a uniqueness constraint.
- **Database Relationships:** Related patient and household records should be associated correctly.
- **Error Handling:** Invalid requests and unsuccessful database operations should be handled appropriately.
- **Data Privacy:** Only authorized and appropriate sample data should be used for academic demonstrations.
- **Export Validation:** Exported CSV data should contain the expected patient and household fields.

8. Testing and Demonstration

The Smart Swasthya application can be tested by checking its major modules and database operations.

1. Open the application dashboard and verify that record counts are displayed.
2. Add and view household information.
3. Add patient information and check whether it is displayed correctly.
4. Add health records containing measurements and notes.
5. Record a patient visit and enter the next follow-up date.
6. Add preventive healthcare activities and verify their status.

7. Open the priority assessment module and check the displayed priority level.
8. Open the reports module and verify the displayed counts.
9. Export patient information to CSV and check the downloaded file.
10. Test invalid inputs and unsuccessful operations to verify appropriate error handling.

For demonstration purposes, use synthetic or sample patient data. Actual patient information should not be entered without appropriate authorization and privacy controls.

9. GitHub Repository and Submission

The complete project source code is maintained in a GitHub repository. The repository contains the application files, styling, database-related SQL file, and Python application code.

GitHub Repository Link:
https://github.com/suyashkarnalemvlu-art/Mern-Project

Project Files:

- `app.py` – Python application and backend processing
- `app.js` – JavaScript functions for table filtering and chart rendering
- `style.css` – CSS styling
- `mysql.sql` – SQL file provided in the repository

Conclusion:

Smart Swasthya provides an organized way to manage community health information, patient visits, preventive activities, follow-ups, and basic reporting. The project demonstrates web application development, database operations, form handling, and rule-based priority assessment.